

Coding Interview Complete Preparation Guide

For Anushka Singhania
Robotics Engineer - Software @ Andromeda Maritime Solutions

Lesson 1: Python Essentials for Interviews

Lesson 2: Arrays & Two Pointers

Lesson 3: Hash Maps (Dictionaries)

Lesson 4: Strings

Lesson 5: Sorting-Based Problem Solving

Lesson 6: Recursion

Lesson 7: BFS, DFS & Graphs

Lesson 8: ROS2 Concepts

Lesson 9: SLAM & Path Planning

Lesson 10: Interview Strategy & Mock Problems

Lesson 1: Python Essentials for Interviews

This is not a full Python course - only the Python you need to write clean code in a 45-minute interview.

1.1 Variables & Data Types

```
# Numbers
x = 10          # int
y = 3.14        # float

# Strings
name = "Anushka"

# Boolean
is_ready = True

# None (like null)
result = None
```

Type conversion (interviewers expect you to know this):

```
int("42")       # string -> int -> 42
str(42)          # int -> string -> "42"
float("3.14")    # string -> float -> 3.14
```

1.2 Lists (Arrays) - Used in EVERY Interview Problem

```
# Creating
nums = [1, 2, 3, 4, 5]
empty = []

# Accessing (0-indexed)
nums[0]        # 1 (first)
nums[-1]       # 5 (last)
nums[-2]       # 4 (second last)

# Slicing
nums[1:3]       # [2, 3] (index 1 to 2, end excluded)
nums[:3]        # [1, 2, 3] (first 3)
nums[2:]       # [3, 4, 5] (from index 2 onward)

# Modifying
nums.append(6)  # add to end
nums.pop()      # remove & return last
nums.insert(0, 99) # insert 99 at index 0
nums.remove(3)  # remove first occurrence of 3

# Length
len(nums)       # how many elements
```

Critical list tricks for interviews:

```

# Sorting
nums.sort()           # sorts in-place (modifies original)
sorted(nums)          # returns NEW sorted list
nums.sort(reverse=True) # descending order

# Reversing
nums.reverse()         # in-place
nums[::-1]            # returns new reversed list

# Check existence, Min, Max, Sum
if 3 in nums: print("found!")
min(nums)
max(nums)
sum(nums)

```

1.3 Dictionaries (Hash Maps) - Your Best Friend

```

# Creating
d = {}
d = {"apple": 3, "banana": 5}

# Adding / Updating
d["cherry"] = 7

# Accessing
d["apple"]          # 3
d.get("mango", 0)   # 0 (default if key missing - SAFER)

# Check if key exists
if "apple" in d:
    print("yes")

# Looping
for key, value in d.items():
    print(key, value)

```

1.4 Loops & Conditions

```

for i in range(5):          # 0, 1, 2, 3, 4
for i in range(2, 8):       # 2, 3, 4, 5, 6, 7
for i in range(0, 10, 2):   # 0, 2, 4, 6, 8 (step=2)

# Loop with index
for i, num in enumerate(nums):
    print(i, num)    # i = index, num = value

# While loop
i = 0
while i < 5:
    print(i)
    i += 1

# If / Elif / Else
if x > 10:    print("big")
elif x > 5:   print("medium")
else:        print("small")

# break = exit loop, continue = skip iteration

```

1.5 Functions

```

def add(a, b):
    return a + b

# Default parameters
def greet(name, msg="Hello"):
    return f"{msg}, {name}!"

# Multiple returns
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([3, 1, 7]) # lo=1, hi=7

```

1.6 Strings & Sets

```

s = "hello world"
s.upper()      # "HELLO WORLD"
s.lower()      # "hello world"
s.split(" ")   # ["hello", "world"]
" ".join(["a","b"])# "a b"
s.strip()      # remove whitespace
s.replace("hello", "hi")
s.find("world") # 6 (index, -1 if not found)
s.count("l")    # 3
f"Hi {name}"    # f-string formatting

# Sets - fast lookup, no duplicates
s = {1, 2, 3}
s.add(4)
unique = list(set([1,2,2,3])) # [1, 2, 3]

```

Lesson 2: Arrays & Two Pointers

TIP: If you see a sorted array or a problem about pairs/subarrays, think Two Pointers first.

2.1 The Brute Force First Rule

Always start with the simplest solution, even if slow. Then optimize. This shows thinking.

Problem: Find two numbers that add up to target. Return their indices.

```
# Brute Force - O(n^2)
def two_sum_brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

2.2 Two Pointers - Opposite Direction

One pointer at start, one at end. Move toward each other. Works on **sorted arrays**.

Example: Reverse an Array

```
def reverse_array(nums):
    left = 0
    right = len(nums) - 1
    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1
    return nums
```

```
[1, 2, 3, 4, 5]
L           R   swap 1 & 5 -> [5, 2, 3, 4, 1]
L     R     swap 2 & 4 -> [5, 4, 3, 2, 1]
    LR      left meets right -> STOP
```

Example: Palindrome Check

```
def is_palindrome(s):
    left = 0
    right = len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True
```

```

"racecar"
r a c e c a r
L           R   r==r  -> move inward
  L         R   a==a  -> move inward
    L      R   c==c  -> move inward
      LR      STOP -> True

```

Example: Two Sum on Sorted Array

```

def two_sum_sorted(nums, target):
    left = 0
    right = len(nums) - 1
    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return [left, right]
        elif current_sum < target:
            left += 1      # need bigger sum
        else:
            right -= 1     # need smaller sum
    return []

```

```

nums = [1, 3, 5, 7, 9], target = 8
left=0, right=4: 1+9=10 > 8 -> right moves to 3
left=0, right=3: 1+7=8 = 8 -> return [0,3]

```

2.3 Two Pointers - Same Direction (Slow/Fast)

Both start at beginning. Fast scans ahead, slow builds result.

Example: Remove Duplicates from Sorted Array

```

def remove_duplicates(nums):
    if len(nums) == 0:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1

```

```

[1, 1, 2, 3, 3, 4]
s f      1==1 same -> skip
s   f    2!=1 DIFF -> slow=1, write 2 -> [1,2,2,3,3,4]
s     f  3!=2 DIFF -> slow=2, write 3 -> [1,2,3,3,3,4]
s       f 3==3 same -> skip
s        f 4!=3 DIFF -> slow=3, write 4 -> [1,2,3,4,_,_]
Return slow+1 = 4

```

Example: Move Zeroes to End

```
def move_zeroes(nums):
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

```
[0, 1, 0, 3, 12]
s f   nums[f]=1 != 0 -> swap -> [1, 0, 0, 3, 12], slow=1
s f   nums[f]=0 -> skip
s   f   nums[f]=3 != 0 -> swap -> [1, 3, 0, 0, 12], slow=2
s       f   nums[f]=12 != 0 -> swap -> [1, 3, 12, 0, 0]
Done!
```

2.4 Sliding Window Pattern

For problems about **contiguous subarrays** of fixed/variable size.

```
# Max sum of k consecutive elements
def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k]) # first window
    max_sum = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k] # slide
        max_sum = max(max_sum, window_sum)
    return max_sum
```

```
[2, 1, 5, 1, 3, 2] k=3
[2, 1, 5]          sum=8
[1, 5, 1]          sum=8-2+1=7
[5, 1, 3]          sum=7-1+3=9 <- max!
[1, 3, 2]          sum=9-5+2=6
Answer: 9
```

2.5 Merge Two Sorted Arrays

```
def merge_sorted(a, b):
    i, j = 0, 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result
```

2.6 Two Pointer Cheat Sheet

Type	Setup	When to Use	Problems
------	-------	-------------	----------

Opposite	left=0, right=end	Sorted array, pairs	Two Sum, Palindrome, Container Water
Slow/Fast	slow=0, fast loops	Remove/filter in-place	Remove Dups, Move Zeroes
Two Arrays	i in A, j in B	Merging sorted data	Merge Sorted Arrays
Sliding Window	fixed/variable window	Contiguous subarray	Max Sum, Longest Substring

Lesson 3: Hash Maps (Dictionaries)

TIP: ~40-50% of easy-medium interview problems can be solved with a dictionary.

3.1 What is a Hash Map?

A magic notebook - store info and look it up **instantly** ($O(1)$).

Regular list: "Is 7 here?" -> check every element -> SLOW $O(n)$
Hash map: "Is 7 here?" -> instant answer -> FAST $O(1)$

3.2 Counting Pattern (Most Common)

```
# Count character frequency
def count_chars(s):
    count = {}
    for char in s:
        count[char] = count.get(char, 0) + 1
    return count

count_chars("banana") # {'b':1, 'a':3, 'n':2}
```

```
"banana"
char='b': {'b':1}
char='a': {'b':1, 'a':1}
char='n': {'b':1, 'a':1, 'n':1}
char='a': {'b':1, 'a':2, 'n':1}
char='n': {'b':1, 'a':2, 'n':2}
char='a': {'b':1, 'a':3, 'n':2}
```

3.3 First Non-Repeating Character

```
def first_unique(s):
    count = {}
    for char in s:
        count[char] = count.get(char, 0) + 1
    for char in s:
        if count[char] == 1:
            return char
    return ""
```

```
"aabbccdd" -> count: {a:2, b:2, c:1, d:2}
Pass 2: a->2 skip, a->2 skip, b->2 skip, b->2 skip, c->1 FOUND!
Return "c"
```

3.4 Two Sum with Hash Map (THE Classic)

```
def two_sum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

```
nums = [2, 7, 11, 15], target = 9
i=0, num=2: comp=7, seen={} -> 7 not found -> seen={2:0}
i=1, num=7: comp=2, seen={2:0} -> 2 FOUND -> return [0,1]
```

3.5 Check Anagrams

```
def is_anagram(s1, s2):
    if len(s1) != len(s2): return False
    count = {}
    for char in s1:
        count[char] = count.get(char, 0) + 1
    for char in s2:
        count[char] = count.get(char, 0) - 1
    return all(v == 0 for v in count.values())
```

3.6 Group Anagrams (Medium - Impressive!)

```
def group_anagrams(words):
    groups = {}
    for word in words:
        key = "".join(sorted(word)) # sort letters = key
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())

# ["eat", "tea", "tan", "ate", "nat", "bat"]
# -> [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

3.7 Majority Element

```
def majority_element(nums):
    count = {}
    threshold = len(nums) / 2
    for num in nums:
        count[num] = count.get(num, 0) + 1
        if count[num] > threshold:
            return num
```

3.8 Hash Map Cheat Sheet

Problem Says...

Hash Map Stores...

Count frequency	element -> count
Find duplicate	element -> True/False (or use set)
Two sum / find pair	value -> index
Group similar things	key -> list of items
First/last occurrence	element -> index
Check if seen before	element -> anything

Lesson 4: Strings

4.1 Reverse a String

```
# Method 1: Slicing (Pythonic)
s = "hello"
reversed_s = s[::-1]    # "olleh"

# Method 2: Two Pointers (show in interview)
def reverse_string(s):
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1
    return "".join(chars)
```

4.2 Valid Palindrome (with cleaning)

Ignore spaces, punctuation, and case.

```
def is_valid_palindrome(s):
    # Keep only letters and digits, lowercase
    cleaned = ""
    for char in s:
        if char.isalnum():      # is letter or digit?
            cleaned += char.lower()

    # Two pointer check
    left, right = 0, len(cleaned) - 1
    while left < right:
        if cleaned[left] != cleaned[right]:
            return False
        left += 1
        right -= 1
    return True

is_valid_palindrome("A man, a plan, a canal: Panama") # True
```

4.3 Longest Substring Without Repeating Characters

Uses **Sliding Window + Hash Map** together.

```
def longest_unique_substring(s):
    seen = {}          # char -> last index
    left = 0
    max_len = 0

    for right in range(len(s)):
        if s[right] in seen and seen[s[right]] >= left:
            left = seen[s[right]] + 1 # shrink window
        seen[s[right]] = right
        max_len = max(max_len, right - left + 1)

    return max_len
```

```
s = "abcabcbb"
r=0 'a': seen={a:0}, window="a", len=1
r=1 'b': seen={a:0,b:1}, window="ab", len=2
r=2 'c': seen={a:0,b:1,c:2}, window="abc", len=3
r=3 'a': a seen at 0, left=1, window="bca", len=3
r=4 'b': b seen at 1, left=2, window="cab", len=3
... Answer: 3 ("abc")
```

4.4 String Compression

```
# "aabcccccaaa" -> "a2b1c5a3"
def compress(s):
    result = ""
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result += s[i-1] + str(count)
            count = 1
    result += s[-1] + str(count) # last group
    return result if len(result) < len(s) else s
```

4.5 Common String Interview Tricks

```
# Check if string has all unique characters
def all_unique(s):
    return len(set(s)) == len(s)

# Count words
words = "hello world foo".split() # ["hello","world","foo"]
len(words) # 3

# Check if one string is rotation of another
# "waterbottle" is rotation of "erbottlewat"
def is_rotation(s1, s2):
    return len(s1) == len(s2) and s2 in s1 + s1
```

Lesson 5: Sorting-Based Problem Solving

TIP: Sorting as a preprocessing step can turn $O(n^2)$ problems into $O(n \log n)$.

5.1 Know Your Sorts

```
# Python built-in sort is Timsort -  $O(n \log n)$ 
nums = [3, 1, 4, 1, 5]
nums.sort()           # in-place: [1, 1, 3, 4, 5]
sorted_nums = sorted(nums) # new list

# Sort by custom key
words = ["banana", "pie", "apple"]
words.sort(key=len)    # by length: ["pie", "apple", "banana"]
words.sort(key=lambda x: x[-1]) # by last char

# Sort tuples
intervals = [(3,5), (1,3), (2,4)]
intervals.sort()        # by first element: [(1,3), (2,4), (3,5)]
intervals.sort(key=lambda x: x[1]) # by second element
```

5.2 Find if Array Has Duplicates

```
# Method 1: Set ( $O(n)$  time,  $O(n)$  space)
def has_duplicates(nums):
    return len(nums) != len(set(nums))

# Method 2: Sort first ( $O(n \log n)$  time,  $O(1)$  space)
def has_duplicates_sort(nums):
    nums.sort()
    for i in range(1, len(nums)):
        if nums[i] == nums[i-1]:
            return True
    return False
```

5.3 Merge Overlapping Intervals (Very Popular)

```
# Input: [(1,3), (2,6), (8,10), (15,18)]
# Output: [(1,6), (8,10), (15,18)]

def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0]) # sort by start
    merged = [intervals[0]]

    for start, end in intervals[1:]:
        last_end = merged[-1][1]
        if start <= last_end: # overlap!
            merged[-1] = (merged[-1][0], max(last_end, end))
        else:
            merged.append((start, end))

    return merged
```

```
Sorted: [(1,3), (2,6), (8,10), (15,18)]
Start with merged = [(1,3)]
(2,6): 2 <= 3? YES overlap -> merge -> [(1,6)]
(8,10): 8 <= 6? NO -> append -> [(1,6), (8,10)]
(15,18): 15 <= 10? NO -> append -> [(1,6), (8,10), (15,18)]
```

5.4 Kth Largest Element

```
# Simple approach: sort and index
def kth_largest(nums, k):
    nums.sort(reverse=True) # descending
    return nums[k - 1] # k=1 -> index 0

# kth_largest([3,2,1,5,6,4], 2) -> 5
```

Lesson 6: Recursion

A function that calls itself. Every recursion needs: (1) Base case - when to stop, (2) Recursive case - break problem into smaller version.

6.1 Factorial - The Classic

```
def factorial(n):
    if n <= 1:          # BASE CASE
        return 1
    return n * factorial(n - 1) # RECURSIVE CASE
```

```
factorial(4)
-> 4 * factorial(3)
-> 4 * (3 * factorial(2))
-> 4 * (3 * (2 * factorial(1)))
-> 4 * (3 * (2 * 1))          # base case hit!
-> 4 * (3 * 2)
-> 4 * 6
-> 24
```

6.2 Fibonacci

```
# 0, 1, 1, 2, 3, 5, 8, 13, 21...
# Each number = sum of previous two

def fib(n):
    if n <= 0: return 0    # base
    if n == 1: return 1    # base
    return fib(n-1) + fib(n-2)

# BETTER: iterative (avoids repeated work)
def fib_fast(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

6.3 Sum of a List (Recursion thinking)

```
def sum_list(nums):
    if len(nums) == 0:    # base: empty list
        return 0
    return nums[0] + sum_list(nums[1:]) # first + rest
```



```

sum_list([3, 1, 4])
-> 3 + sum_list([1, 4])
-> 3 + (1 + sum_list([4]))
-> 3 + (1 + (4 + sum_list([])))
-> 3 + (1 + (4 + 0))    # base case!
-> 3 + (1 + 4) -> 3 + 5 -> 8

```

6.4 Binary Search (Recursion + Interviews)

Search a sorted array in $O(\log n)$ by halving the search space.

```

# Iterative (preferred in interviews)
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1 # not found

```

```

nums = [1, 3, 5, 7, 9, 11], target = 7
left=0, right=5, mid=2: nums[2]=5 < 7 -> left=3
left=3, right=5, mid=4: nums[4]=9 > 7 -> right=3
left=3, right=3, mid=3: nums[3]=7 == 7 -> return 3!

```

6.5 Power of a Number

```

def power(base, exp):
    if exp == 0: return 1          # anything^0 = 1
    if exp == 1: return base      # base^1 = base
    half = power(base, exp // 2)
    if exp % 2 == 0:
        return half * half        # even: x^4 = x^2 * x^2
    else:
        return half * half * base # odd: x^5 = x^2 * x^2 * x

```

6.6 Recursion Template

```

def solve(problem):
    if base_case(problem):      # 1. When to STOP
        return base_answer

    smaller = break_down(problem) # 2. Make it SMALLER
    result = solve(smaller)       # 3. Solve the smaller version
    return combine(result)        # 4. Build the answer

```

Lesson 7: BFS, DFS & Graphs

TIP: BFS/DFS on graphs = Path Planning in robotics. A*, Dijkstra are extensions of BFS.

7.1 Graph Basics

A graph = nodes (vertices) + edges (connections). Stored as adjacency list:

```
# Graph as dictionary
graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"]
}
# "A" connects to "B" and "C"
# "B" connects to "A", "D", "E" etc.
```

7.2 BFS - Breadth First Search (Level by Level)

Uses a **queue**. Explores all neighbors first, then their neighbors. Finds **shortest path**.

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    result = []

    while queue:
        node = queue.popleft()    # take from front
        result.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)    # add to back

    return result
```

```
bfs(graph, "A")
Queue: [A]      Visit A -> neighbors B,C
Queue: [B, C]   Visit B -> neighbors D,E
Queue: [C, D, E] Visit C -> neighbor F
Queue: [D, E, F] Visit D -> no new
Queue: [E, F]   Visit E -> no new
Queue: [F]      Visit F -> no new
Result: [A, B, C, D, E, F] (level by level)
```

7.3 DFS - Depth First Search (Go Deep First)

Uses a **stack** (or recursion). Goes as deep as possible before backtracking.

```
# Iterative DFS (using stack)
def dfs(graph, start):
    visited = set()
    stack = [start]
    result = []

    while stack:
        node = stack.pop()          # take from top
        if node not in visited:
            visited.add(node)
            result.append(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)

    return result

# Recursive DFS
def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)
```

7.4 BFS Shortest Path (Very Important for Robotics)

```
def shortest_path(graph, start, end):
    queue = deque([(start, [start])]) # (node, path)
    visited = set([start])

    while queue:
        node, path = queue.popleft()

        if node == end:
            return path          # found shortest!

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None # no path exists
```

```
shortest_path(graph, "A", "F")
Queue: [(A, [A])]          Visit A
Queue: [(B, [A,B]), (C, [A,C])] Visit B
Queue: [(C, [A,C]), (D, [A,B,D]), (E, [A,B,E])] Visit C
-> C has neighbor F!
Queue: [..., (F, [A,C,F])] Visit F = target!
Return [A, C, F] (shortest path, length 2)
```

7.5 Grid BFS (Robot Navigation!)

Treat grid as graph. Each cell connects to 4 neighbors (up/down/left/right).

```
def grid_bfs(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    queue = deque([(start[0], start[1], 0)]) # row, col, distance
    visited = set()
    visited.add((start[0], start[1]))

    directions = [(0,1), (0,-1), (1,0), (-1,0)] # R,L,D,U

    while queue:
        row, col, dist = queue.popleft()

        if (row, col) == end:
            return dist

        for dr, dc in directions:
            new_r, new_c = row + dr, col + dc
            if (0 <= new_r < rows and 0 <= new_c < cols
                and grid[new_r][new_c] != 1 # not wall
                and (new_r, new_c) not in visited):
                visited.add((new_r, new_c))
                queue.append((new_r, new_c, dist + 1))

    return -1 # no path
```

```
grid = [
    [0, 0, 0, 0],    0 = free, 1 = wall
    [0, 1, 1, 0],    S = start (0,0)
    [0, 0, 0, 0],    E = end (2,3)
]
BFS explores level by level:
Dist 0: (0,0)
Dist 1: (0,1), (1,0)
Dist 2: (0,2), (2,0)
Dist 3: (0,3), (2,1)
Dist 4: (2,2)
Dist 5: (2,3) = END! Shortest path = 5
```

7.6 BFS vs DFS Cheat Sheet

	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO) / Recursion
Explores	Level by level	As deep as possible
Best For	Shortest path	Explore all paths, detect cycles
Space	O(width)	O(depth)
Robotics Use	Path planning, wavefront	Maze solving, mapping

Lesson 8: ROS2 Concepts Rapid Review

TIP: You used ROS2 + MoveIt2 in your thesis. Refresh, don't relearn. Own your experience.

8.1 Core Concepts

Concept	What It Is	Analogy
Node	A process that does one thing	A worker in a factory
Topic	Named channel for streaming data	A radio frequency
Publisher	Sends messages to a topic	Radio broadcaster
Subscriber	Listens to a topic	Radio listener
Service	Request-reply (like a function call)	Calling customer support
Action	Long-running task with feedback	Ordering food delivery + tracking
Launch File	Starts multiple nodes at once	A startup script
Parameter	Configuration value for a node	Settings in an app
tf2	Tracks coordinate transforms	GPS for robot parts
Bag	Records all topic data	Flight recorder / black box

8.2 Topic vs Service vs Action

Topic (Publisher/Subscriber):

- One-way, continuous stream
- Example: /camera/image publishes frames at 30Hz
- No response expected

Service (Client/Server):

- Two-way, one-time request-reply
- Example: /get_map_data -> returns map
- Blocks until response

Action (Client/Server + Feedback):

- Long-running with progress updates
- Example: /navigate_to_waypoint -> "50% there... 80%... done!"
- Can be cancelled mid-way

8.3 Minimal ROS2 Python Node

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MyPublisher(Node):
    def __init__(self):
        super().__init__("my_publisher")
        self.publisher = self.create_publisher(String, "my_topic", 10)
        self.timer = self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        msg = String()
        msg.data = "Hello from ROS2!"
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = MyPublisher()
    rclpy.spin(node)
    rclpy.shutdown()

```

8.4 Key ROS2 Commands

```

ros2 topic list           # see all active topics
ros2 topic echo /topic    # see messages on a topic
ros2 node list            # see all running nodes
ros2 service list         # see all services
ros2 launch pkg file.py   # run a launch file
ros2 bag record -a        # record all topics
ros2 param list           # list all parameters

```

8.5 YOUR Thesis Pipeline (know this cold)

Your pipeline: Camera -> Point Cloud -> GraspGen -> MoveIt2 -> UR10

1. Intel RealSense D435 captures RGB-D images
2. Point cloud generated from depth data
3. GraspGen (diffusion model) generates grasp poses
4. MoveIt2 plans collision-free motion
5. UR10 executes the grasp

Key ROS2 concepts you used:

- Topics: camera data, point clouds, grasp poses
- tf2: camera frame -> robot base frame transform
- MoveIt2: motion planning as an Action
- Launch files: starting entire pipeline

Lesson 9: SLAM & Path Planning

TIP: For underwater robots - no GPS. Localization uses DVL + IMU + sonar instead.

9.1 What is SLAM?

Simultaneous Localization And Mapping - the robot figures out WHERE it is (localization) while building a MAP of its environment, at the same time. Chicken-and-egg problem: need a map to know location, need location to build a map.

Type	How It Works	Used With
EKF-SLAM	Extended Kalman Filter, tracks landmarks	Small environments, few landmarks
Particle Filter SLAM	Many random guesses, keep best ones	Large environments
Graph SLAM	Poses as nodes, constraints as edges, optimize graph	Most modern systems
Visual SLAM	Uses camera images for features	ORB-SLAM, cameras
LiDAR SLAM	Uses laser scans for mapping	Hector SLAM, Gmapping

9.2 Underwater Robotics Context (Andromeda Specific)

Underwater challenges:

- No GPS signal underwater
- No reliable radio communication
- Limited visibility (murky water)
- Pressure increases with depth

Sensors used:

- DVL (Doppler Velocity Log): measures velocity relative to seabed
- IMU: acceleration + rotation
- Depth sensor: pressure-based depth
- Sonar: acoustic ranging (like underwater LiDAR)
- Cameras: visual data (limited range underwater)

Localization approach:

- Dead reckoning: DVL + IMU integration
- Acoustic positioning: USBL/LBL beacons
- Surface GPS fix when vehicle surfaces

9.3 Path Planning Algorithms

A* (A-Star) - The Gold Standard

BFS + heuristic. Explores promising directions first.

```

import heapq

def a_star(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    # Priority queue: (f_score, row, col, path)
    # f = g + h (actual cost + estimated remaining)
    heap = [(0, start[0], start[1], [start])]
    visited = set()

    def heuristic(r, c):
        return abs(r - end[0]) + abs(c - end[1]) # Manhattan

    while heap:
        f, r, c, path = heapq.heappop(heap)

        if (r, c) == end:
            return path

        if (r, c) in visited:
            continue
        visited.add((r, c))

        g = len(path) # actual cost so far
        for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc = r + dr, c + dc
            if (0<=nr<rows and 0<=nc<cols
                and grid[nr][nc] != 1
                and (nr,nc) not in visited):
                new_f = g + 1 + heuristic(nr, nc)
                heapq.heappush(heap,
                    (new_f, nr, nc, path+[(nr,nc)]))

    return None

```

Dijkstra's Algorithm

Like A* but without heuristic. Guarantees shortest path in weighted graphs.

```

# Dijkstra = A* where heuristic = 0
# Use when: edges have different weights/costs
# Example: road network where roads have different lengths

```

Key difference:

```

BFS:      all edges weight 1, uses queue
Dijkstra: edges have weights, uses priority queue
A*:       Dijkstra + heuristic for faster search

```

RRT (Rapidly-exploring Random Trees)

RRT: Random sampling-based planner

1. Pick a random point in space
2. Find nearest node in tree
3. Extend tree toward random point
4. Check for collisions
5. Repeat until goal reached

RRT*: Improved RRT that rewires tree for optimal paths

Used in: robot arm motion planning (like MoveIt2!),
high-dimensional spaces, underwater navigation

9.4 Algorithm Comparison

Algorithm	Optimal?	Complete?	Best For
BFS	Yes (unweighted)	Yes	Grid, equal costs
Dijkstra	Yes	Yes	Weighted graphs
A*	Yes (good h)	Yes	Weighted + heuristic
RRT	No	Probabilistic	High-dim, complex
RRT*	Asymptotic	Probabilistic	Optimal + complex

Lesson 10: Interview Strategy & Mock Problems

10.1 The Interview Framework (Follow This Every Time)

STEP 1: UNDERSTAND (2 min)

- Repeat the problem in your own words
- Ask clarifying questions
- Work through the given example by hand

STEP 2: PLAN (3 min)

- State brute force approach and its time complexity
- Propose optimized approach
- Explain WHY it's better (what pattern you're using)

STEP 3: CODE (10-15 min)

- Write clean code with good variable names
- Talk while coding ("now I'm iterating through...")
- Don't worry about perfection

STEP 4: TEST (3 min)

- Trace through your code with the example
- Think of edge cases: empty input, one element, duplicates
- Fix any bugs you find

10.2 What to Say When Stuck

GOOD things to say:

"Let me think about what data structure fits here..."
"I'm considering two approaches: X and Y..."
"This reminds me of the [pattern] pattern..."
"Can I sort this first to make it easier?"
"What if I used a hash map to track what I've seen?"
"Let me start with brute force and optimize after."

BAD things to do:

- Going silent for more than 30 seconds
- Jumping straight to coding without a plan
- Saying "I don't know" and giving up
- Writing code without explaining what you're doing

10.3 Pattern Recognition Quick Reference

If You See...	Think...	Pattern
Sorted array + pair	Two pointers	Opposite direction
Remove/filter in-place	Two pointers	Slow/Fast
Subarray of size k	Sliding window	Fixed window
Longest/shortest subarray	Sliding window	Variable window

Count/frequency	Hash map	Counting
Find pair summing to X	Hash map	Complement lookup
Shortest path (grid)	BFS	Queue + visited
Explore all possibilities	DFS/Recursion	Stack/backtrack
Optimal path (weighted)	Dijkstra / A*	Priority queue

10.4 Mock Problem Walkthrough

Problem: "Given a grid where 0 = water and 1 = land, count the number of islands."

Step 1 - Understand:

```
grid = [
    [1, 1, 0, 0, 0],
    [1, 1, 0, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 1, 1]
]
```

Answer: 3 islands (top-left block, middle, bottom-right)

Step 2 - Plan: Scan grid. When I find a 1, that's a new island. Use DFS/BFS to mark all connected 1s as visited. Count how many times I start a new search.

Step 3 - Code:

```
def count_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    count = 0

    def dfs(r, c):
        if (r < 0 or r >= rows or c < 0 or c >= cols
            or grid[r][c] == 0):
            return
        grid[r][c] = 0 # mark visited (sink the land)
        dfs(r+1, c)    # down
        dfs(r-1, c)    # up
        dfs(r, c+1)    # right
        dfs(r, c-1)    # left

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                count += 1 # found new island!
                dfs(r, c) # mark entire island

    return count
```

Step 4 - Test: Walk through the grid, find first 1 at (0,0), DFS marks all connected land, count=1. Next 1 at (2,2), DFS marks it, count=2. Next 1 at (3,3), DFS marks (3,3) and (3,4), count=3. Return 3.

10.5 Time Complexity Quick Reference

Notation	Name	Example	Speed
$O(1)$	Constant	Hash map lookup	Fastest
$O(\log n)$	Logarithmic	Binary search	Very fast
$O(n)$	Linear	Single loop	Fast
$O(n \log n)$	Log-linear	Sorting	Good
$O(n^2)$	Quadratic	Nested loops	Slow
$O(2^n)$	Exponential	All subsets	Very slow

You've got this, Anushka. The hardest part - getting shortlisted - is already done.

Own your thesis. Think out loud. Start brute force, then optimize.